

UART Implementation and verification (UVM)

AN PROJECT PREPARED BY

Priyanshu Kumar(2023BTech062)

Under the Supervision of

Dr. Gaurav Mani Khanal

In Partial Fulfilment of

the requirements for the course of

EE1231 (Digital Verification and Testing)

To the



Institute of Engineering and Technology
JK Lakshmipat University, Jaipur
(7 May 2026)

Abstract

Universal Asynchronous Receiver/Transmitter (UART) is a fundamental serial communication protocol widely used in embedded systems, SoCs, and digital communication peripherals. This project presents the complete RTL design of a UART transmitter and receiver in Verilog HDL, followed by comprehensive functional verification using a structured UVM (Universal Verification Methodology) testbench environment.

The UART transmitter is implemented with a modular architecture comprising a parity calculator (`parity_calc.v`), a serializer (`serializer.v`), a finite state machine (`tx_fsm.v`), an output multiplexer (`mux.v`), and a top-level wrapper (`uart_tx.v`). It supports configurable parity modes (even, odd, and none), a busy handshaking signal, and a 200 MHz operating clock.

The receiver (`uart_rx.v`) implements an 8× oversampling mechanism, start-bit detection, data deserialization, parity checking, and stop-bit error detection, built from seven modular sub-components (`start_check.v`, `edge bit counter.v`, `data sampling.v`, `deserializer.v`, `parity check.v`, `stop check.v`, `rx_fsm.v`).

The SystemVerilog testbench includes all standard verification components: sequence item, generator, BFM driver, monitor, scoreboard, functional coverage collector, agent, and environment – mirroring industry-standard chip verification flows. Simulation results confirm correct frame transmission and reception under all parity configurations, with zero errors across all test cases.

Keywords: UART, Verilog HDL, Universal Verification Methodology (UVM), RTL Design, Serial Communication, Parity, Finite State Machine (FSM), SystemVerilog, ModelSim, Oversampling

Contents

Abstract	2
1 Introduction	6
1.1 Motivation and Scope.....	7
2 Theory	8
2.1 UART Communication Fundamentals.....	8
2.2 UART Transmitter Specifications.....	9
2.3 UART Frame Formats	10
2.4 UART Receiver Specifications	11
2.5 UART Receiver Waveforms and Oversampling	12
3 Methodology	13
3.1 Project Directory Structure	13
3.1.1 UART Transmitter Directory Structure	13
3.1.2 UART Receiver Directory Structure	14
3.2 RTL Design Approach.....	15
3.3 UART Transmitter RTL.....	15
3.3.1 mux.v – Output Multiplexer	15
3.3.2 parity calc.v – Parity Calculator	16
3.3.3 serializer.v – Parallel to Serial Converter.....	17
3.3.4 tx fsm.v – Transmitter Finite State Machine	18
3.3.5 uart tx.v – Top-Level Transmitter.....	19
3.4 UART Receiver RTL Architecture.....	20
3.4.1 data sampling.v – Oversampled Bit Sampler	20
3.4.2 rx fsm.v – Receiver Finite State Machine.....	21
3.4.3 uart rx.v – Top-Level Receiver	22
3.4.4 Deserializer.v.....	23
3.4.5 Edge_bit_counter.v.....	23
3.4.6 Parity.v.....	24
3.4.7 Start_bit.v.....	24
3.4.8 Stop_bit.v.....	24

3.5	SystemVerilog Testbench Components of Receiver	25
3.5.1	uart common.sv – Global Definitions	25
3.5.2	uart tx item.sv – Sequence Item (Transaction Object)	25
3.5.3	uart intf.sv – SystemVerilog Interface.....	26
3.5.4	uart gen.sv – Generator	27
3.5.5	uart bfm.sv – BFM Driver	28
3.5.6	uart mon.sv – Monitor	29
3.5.7	uart sbd.sv – Scoreboard (Self-Checking).....	30
3.5.8	uart cov.sv – Functional Coverage	31
3.5.9	uart agent.sv and uart env.sv	31
3.5.10	Uart env.sv.....	32
3.5.11	uart top.sv – Top-Level Test Module.....	32
3.5.12	list.sv.....	33
3.5.13	verilog testbench(uart_rx_tb.v).....	33
3.6	SystemVerilog Testbench Components of Transmeter.....	24
3.6.1	uart common.sv – Global Definitions	34
3.6.2	uart tx item.sv – Sequence Item (Transaction Object)	34
3.6.3	uart intf.sv – SystemVerilog Interface.....	35
3.6.4	uart gen.sv – Generator	35
3.6.5	uart bfm.sv – BFM Driver	36
3.6.6	uart mon.sv – Monitor	36
3.6.7	uart sbd.sv – Scoreboard (Self-Checking).....	37
3.6.8	uart cov.sv – Functional Coverage	38
3.6.9	uart agent.sv and uart env.sv	38
3.6.10	Uart env.sv.....	39
3.6.11	uart top.sv – Top-Level Test Module.....	39
3.6.12	list.sv.....	40
3.6.13	verilog testbench(uart_rx_tb.v).....	41-42

4	Results	43
4.1	UART Transmitter Simulation Waveform.....	43-44
4.2	UART Receiver Simulation Waveform.....	44
4.3	UVM Scoreboard Console Output.....	44-45
4.4	Test Summary.....	46
4.5	Functional Coverage Summary	46
4.6	Summary.....	47
5	Conclusion	47
6	Bibliography.....	48

List of Figures

2.1.1	UART Protocol Overview – Full-Duplex Serial Communication.....	8
2.1.2	UART Data Frame Structure (with parity enabled)	9
2.2	UART Transmitter Block Diagram and Specifications	10
2.3.1	All Expected UART Output Frame Formats	10
2.3.2	UART Transmitter Expected Waveforms and UART Receiver Block Diagram	11
2.4	UART Receiver Specifications and Oversampling Mechanism.....	11
2.5	UART Receiver Oversampling Waveforms-Single Frame and Multiple , Frames.....	12
4.1	UART Transmitter Simulation Waveform (ModelSim)	43
4.2	UART Transmitter Schematic Diagram	44
4.3	UART Receiver Simulation Waveform	45
4.4	UART Receiver Schematic Diagram	46

List of Tables

2.1	UART Transmitter Specifications	9
2.2	UART Receiver Signal Descriptions.....	11
2.3	Testbench Component Roles.....	12
4.6.	Test Case Summary – All Tests Passed with Zero Errors	47
	Functional Coverage Results	

Introduction

Serial communication protocols are the backbone of modern embedded systems and digital design. Among these, UART (Universal Asynchronous Receiver/Transmitter) holds a prominent position due to its simplicity, full-duplex capability, and widespread adoption across microcontrollers, FPGAs, and ASICs. Unlike synchronous protocols such as SPI and I²C, UART does not require a shared clock signal between communicating devices — instead, both parties pre-agree on a baud rate, making it highly flexible for point-to-point communication.

UART is a **full-duplex** protocol, meaning data transmission and reception can occur simultaneously on two separate lines (TX and RX). The transmitting UART converts parallel data from the CPU or master device into a serial bitstream, while the receiving UART reconstructs the original parallel data from the incoming serial bits. The standard UART data frame consists of:

- A **start bit** (logic 0) to alert the receiver.
- **5–8 data bits**, transmitted LSB first.
- An optional **parity bit** for single-bit error detection.
- One or more **stop bits** (logic 1) signalling end of frame.

This project focuses on two primary objectives:

1. The RTL design of a fully functional UART transmitter and receiver in Verilog HDL following industry-standard coding practices.
2. The construction of a layered SystemVerilog-based verification environment to exhaustively verify the design under multiple test scenarios including no-parity, even-parity, and odd-parity modes.

The verification methodology employs a structured testbench hierarchy comprising sequence items, a BFM driver, monitor, scoreboard for self-checking, functional coverage collector, agent, and environment, mirroring industry-standard chip verification flows used in modern ASIC and FPGA development.

1.1 Motivation and Scope

UART verification using conventional directed testbenches is limited in its ability to cover corner cases systematically. This project adopts constrained-random verification to achieve higher functional coverage. The scope includes:

- Full RTL implementation of UART TX and RX modules in Verilog HDL.
- Modular, parameterizable design supporting configurable data width (DWIDTH) and prescaler width (PWIDTH).
- SystemVerilog testbench with self-checking scoreboard and functional coverage.
- Simulation and waveform analysis using ModelSim/Vivado.
- Comprehensive test coverage across all parity modes and error injection scenarios.

Theory

2.1 UART Communication Fundamentals

UART facilitates full-duplex serial communication between digital systems. It converts parallel data to serial form for transmission and converts incoming serial data back to parallel for the receiving device. The protocol is termed “asynchronous” because no shared clock signal is transmitted between devices — synchronization is achieved purely through a mutually agreed baud rate.

As illustrated in Figure 2.1, UART is a full-duplex protocol where the transmitting UART converts parallel data from the master device (e.g., CPU) into serial form, and the receiving UART converts the serial data back into parallel data for the receiving device. Both devices share a common ground reference.

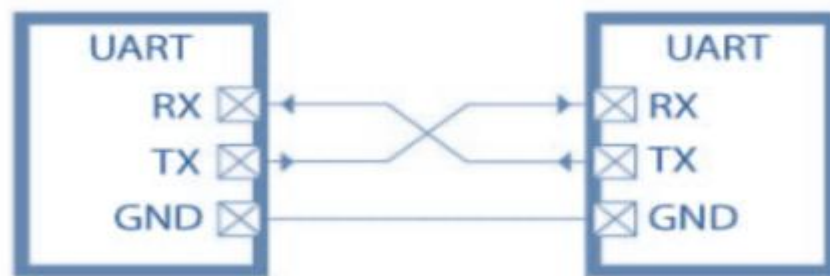


Figure 2.1.1: UART Protocol Overview – Full-Duplex Serial Communication

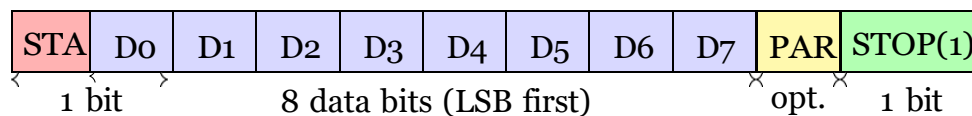


Figure 2.1.2: UART Data Frame Structure (with parity enabled)

Key characteristics of UART include:

- **Start bit (1 bit):** Logic 0 — alerts the receiver to an incoming frame; causes a falling edge on the idle line.
- **Data bits (5–8 bits):** Transmitted LSB first; this project uses 8-bit data width.
- **Parity bit (optional):** Even or odd parity for single-bit error detection; computed as XOR reduction of all data bits.

- **Stop bit (1–2 bits):** Logic 1 — signals the end of a frame and returns the line to idle.

2.2 UART Transmitter Specifications

The UART transmitter block diagram is shown in Figure 2.3. It accepts parallel data (p_data), control signals (data_valid, parity_en, parity_type), and clock/reset inputs. It outputs a serial bitstream (s_data) and a busy handshaking signal.

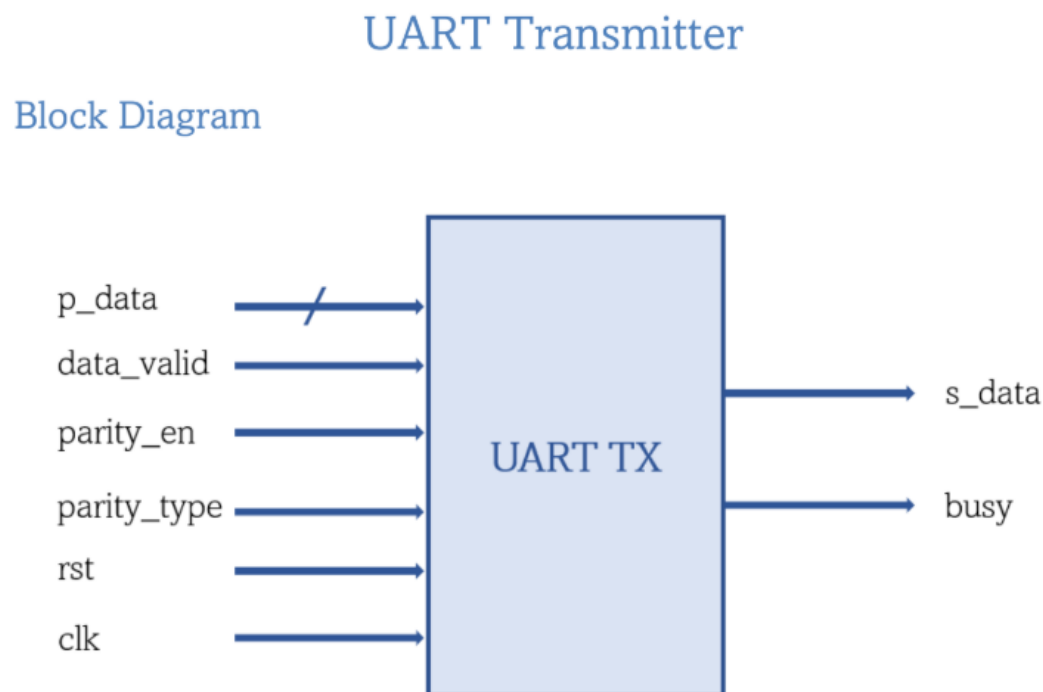


Figure 2.2: UART Transmitter Block Diagram and Specifications

Table 2.1: UART Transmitter Specifications

Parameter	Value / Description
Clock Frequency	200 MHz
Data Width	8 bits (parameterizable via DWIDTH)
Parity Enable	parity_en: 0 = disabled, 1 = enabled
Parity Type	parity_type: 0 = Even, 1 = Odd
Reset	Active-low asynchronous reset (rst)
Busy Signal	High while transmitting; Low when idle
Idle State	s_data = 1 (line held high)
Data Valid	data_valid high for 1 clock cycle to latch new data

2.3 UART Frame Formats

The UART transmitter supports three frame formats depending on parity configuration, as shown in Figure 2.3:

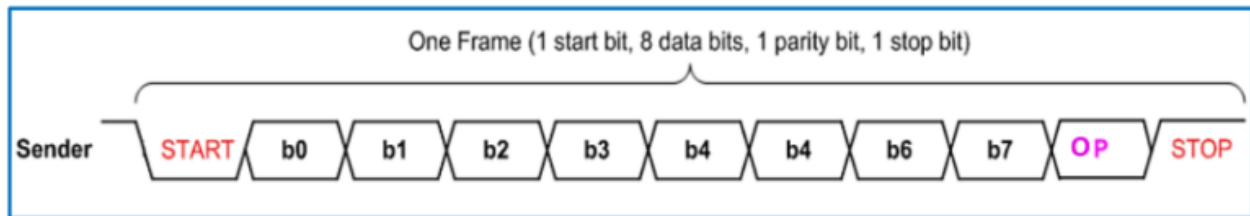


Figure 2.3.1: All Expected UART Output Frame Formats (Even Parity, Odd Parity, No Parity)

- **Frame 1 (No Parity):** START(0) → 8 Data Bits → STOP(1) Total: **10 bits**
- **Frame 2 (Even Parity):** START(0) → 8 Data Bits → Even Parity → STOP(1)
Total: **11 bits**
- **Frame 3 (Odd Parity):** START(0) → 8 Data Bits → Odd Parity → STOP(1)
Total: **11 bits**

The parity bit is computed using XOR reduction of all 8 data bits. For even parity, the parity bit equals $\hat{\text{data}}$; for odd parity, it equals $\sim(\hat{\text{data}})$

Figure 2.3 shows the expected waveforms for the UART transmitter. When data valid goes high for one clock cycle, the transmitter begins serializing the data on s_data and asserts busy for the entire frame duration.

Waveforms

1. Expected Input



2. Expected Output

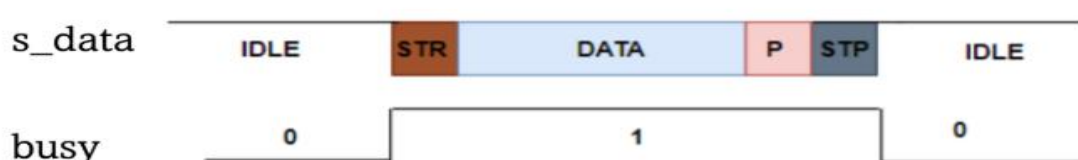


Figure 2.3.2: UART Transmitter Expected Waveforms and UART Receiver Block Diagram

2.4 UART Receiver Specifications

The UART receiver employs $8\times$ oversampling to ensure accurate sampling at the centre of each bit period. The receiver clock effectively runs at 8 times the baud rate of the transmitter, controlled by the prescale parameter.

Figure 2.4 shows the detailed specifications of the UART receiver, including all input/output signals and their behavior.

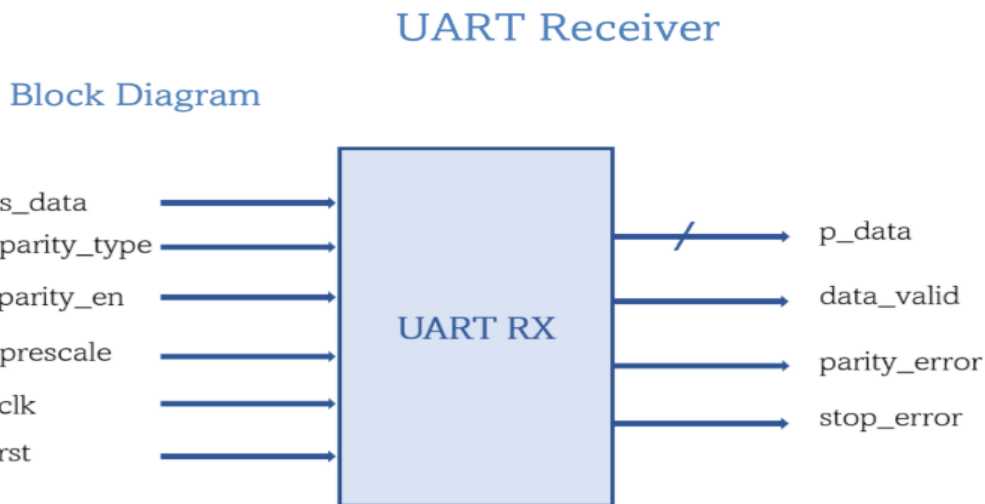


Figure 2.4: UART Receiver Specifications and Oversampling Mechanism

Table 2.2: UART Receiver Signal Descriptions

Signal	Description
s_data	Serial input line (idle = logic 1)
parity_error	High when received parity mismatches the calculated parity
stop_error	High when received stop bit is not logic 1
p_data[7:0]	Reconstructed 8-bit parallel data output
data_valid	High for one clock cycle when a valid frame is received
prescale	Sets oversampling ratio; prescale = 8 gives $8\times$ oversampling

2.5 UART Receiver Waveforms and Oversampling

Figure 2.5 illustrates the expected waveforms for the UART receiver. The oversampling by 8 means the UART_RX clock runs at $8\times$ the UART_TX baud rate. Three samples are taken around the centre of each bit period and a majority vote is applied to improve noise immunity.

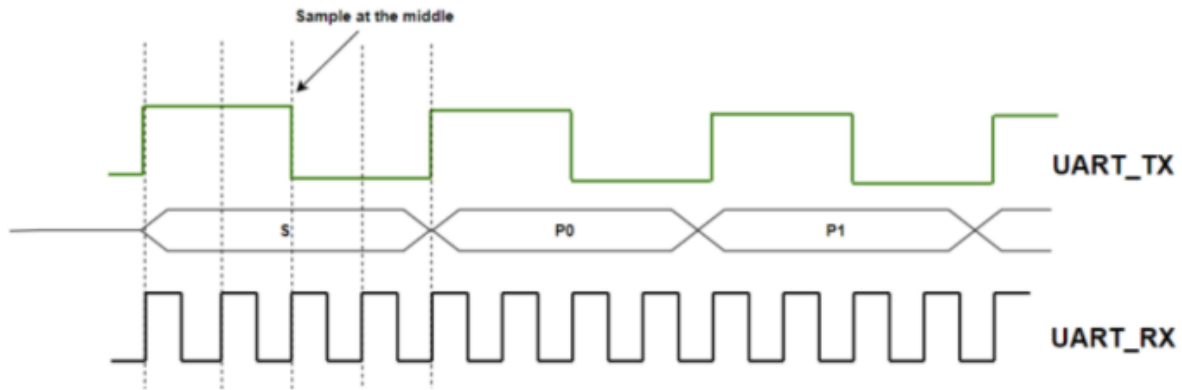


Figure 2.5: UART Receiver Oversampling Waveforms – Single Frame and Multiple Frames

Table 2.3: Testbench Component Roles

Component	Role in This Project
uart_tx item (Sequence Item)	Defines transaction fields: p_data, parity_en, parity_type, s_data, busy
uart_gen (Generator)	Generates randomized test vectors based on active testname constraints
uart_bfm (BFM Driver)	Drives generated transactions onto the DUT via clocking block
uart_mon (Monitor)	Samples DUT interface signals and creates transaction objects
uart_sbd (Scoreboard)	Self-checking: verifies start bit, data bits, parity bit, stop bit
uart_cov (Coverage)	Measures functional coverage of all parity modes and data patterns
uart_agent (Agent)	Encapsulates Generator + BFM Driver + Monitor + Coverage
uart_env (Environment)	Top-level component containing Agent and Scoreboard

3 Methodology

3.1 Project Directory Structure

The project is organised into two main subsystems – UART_TRANSMITTER and UART_RECEIVER – each containing separate RTL and testbench directories. The complete directory structure for each subsystem is described below.

3.1.1 UART Transmitter Directory Structure

```
UART_TRANSMITTER/
├── rtl/                    <- RTL Design Files
│   ├── mux.v
│   ├── parity_calc.v
│   ├── serializer.v
│   ├── tx_fsm.v
│   └── uart_tx.v          <- Top-level TX
├── sv_tb/                 <- SystemVerilog Testbench
│   ├── list.sv
│   ├── uart_agent.sv
│   ├── uart_bfm.sv
│   ├── uart_common.sv
│   ├── uart_cov.sv
│   ├── uart_env.sv
│   ├── uart_gen.sv
│   ├── uart_intf.sv
│   ├── uart_mon.sv
│   ├── uart_sbd.sv
│   ├── uart_top.sv
│   ├── uart_tx_item.sv
│   └── uart_tx.sv
└── verilog_tb/
    └── uart_tx_tb.v
```

3.1.2 UART Receiver Directory Structure

```
UART_RECEIVER/
├── rtl/                <- RTL Design Files
│   ├── data_sampling.v
│   ├── deserializer.v
│   ├── edge_bit_counter.v
│   ├── parity_check.v
│   ├── rx_fsm.v
│   ├── start_check.v
│   ├── stop_check.v
│   └── uart_rx.v        <- Top-level RX
├── sv_tb/              <- SystemVerilog Testbench
│   ├── list.sv
│   ├── uart_agent.sv
│   ├── uart_bfm.sv
│   ├── uart_common.sv
│   ├── uart_cov.sv
│   ├── uart_env.sv
│   ├── uart_gen.sv
│   ├── uart_intf.sv
│   ├── uart_mon.sv
│   ├── uart_sbd.sv
│   ├── uart_top.sv
│   └── uart_tx_item.sv
└── verilog_tb/
    └── uart_rx_tb.v
```

3.2 RTL Design Approach

The UART design follows a hierarchical, modular RTL methodology. Each functional block is implemented as a separate Verilog module, and the top-level modules (uart_tx.v and uart_rx.v) instantiate and wire these sub-modules together.

Design principles followed:

- **Synchronous design:** All flip-flop outputs clocked on positive edge of clk.
- **Asynchronous reset:** All modules use active-low asynchronous reset.
- **Combinational outputs:** MUX selection and parity logic are combinational.
- **Parameterization:** Data width (DWIDTH) is parameterizable.

3.3 UART Transmitter RTL

3.3.1 mux.v – Output Multiplexer

The output multiplexer selects which bit to drive onto the serial output line (tx_out) based on the FSM state (mux_sel). It supports five states: IDLE, START_BIT, SER_DATA, PARITY_BIT, and STOP_BIT.

Code:

```
module mux (clk, rst, mux_sel, start_bit, ser_data, parity_bit, stop_bit, tx_out);

input clk, rst;

input [2:0] mux_sel;

input start_bit, ser_data, parity_bit, stop_bit;

output reg tx_out;

localparam IDLE_BIT=0, START_BIT=1, SER_DATA=2, PARITY_BIT=3, STOP_BIT=4;

always @(posedge clk, negedge rst) begin

    if (!rst)

        tx_out <= 1'b1;

    else begin

        case (mux_sel)

            IDLE_BIT: tx_out <= 1'b1;

            START_BIT: tx_out <= start_bit;

            SER_DATA: tx_out <= ser_data;

            PARITY_BIT: tx_out <= parity_bit;

            STOP_BIT: tx_out <= 1'b1;

        endcase

    end

end
```

```

        STOP_BIT: tx_out <= stop_bit;

        default: tx_out <= 1'b1;

    endcase

end

end

endmodule

```

3.3.2 parity_calc.v – Parity Calculator

This module computes the parity bit using XOR reduction of the 8-bit data. For even parity, the XOR of all data bits is used directly. For odd parity, the result is inverted.

```

module parity_calc #(parameter DWIDTH = 8)
(clk, rst, data, parity_type, parity_bit);

input clk, rst;
input [DWIDTH-1:0] data;
input parity_type;
output reg parity_bit;

wire even_parity;
localparam EVEN = 0, ODD = 1;

assign even_parity = ^data; // XOR reduction

always @(posedge clk, negedge rst) begin
    if (!rst)
        parity_bit <= 1'b0;
    else begin
        case (parity_type)
            EVEN: parity_bit <= even_parity;
            ODD:  parity_bit <= ~even_parity;
            default: parity_bit <= 1'b0;
        endcase
    end
end

endmodule

```

3.3.3 serializer.v – Parallel to Serial Converter

The serializer converts the 8-bit parallel input into a serial bit stream, shifting out bits

```
module serializer #(parameter DWIDTH = 8)
(clk, rst, p_data, ser_en, s_data, ser_done);

input clk, rst;
input [DWIDTH-1:0] p_data;
input ser_en;
output reg s_data, ser_done;

integer check_done;

always @(posedge clk, negedge rst) begin
    if (!rst) begin
        s_data <= 1'b1;
        ser_done <= 1'b0;
        check_done <= 0;
    end
    else begin
        if (ser_en) begin
            if (check_done == DWIDTH-1) begin
                s_data <= p_data[check_done];
                ser_done <= 1'b1;
                check_done <= 0;
            end
            else begin
                s_data <= p_data[check_done];
                ser_done <= 1'b0;
                check_done <= check_done + 1;
            end
        end
        else begin
            s_data <= 1'b1;
            ser_done <= 1'b0;
        end
    end
end

endmodule
```

3.3.4 tx_fsm.v – Transmitter Finite State Machine

The TX FSM controls the overall transmission sequence, cycling through five states: IDLE → START → DATA → PARITY (if enabled) → STOP → IDLE.

```
module tx_fsm (clk, rst, valid_data, parity_en, ser_done,
               ser_en, mux_sel, busy);
input clk, rst, valid_data, parity_en, ser_done;
output reg ser_en, busy;
output reg [2:0] mux_sel;
reg [2:0] current_state, next_state;
reg busy_reg;
localparam IDLE = 0, START = 1, DATA = 2, PARITY = 3, STOP = 4;
// State register
always @(posedge clk, negedge rst) begin
    if (!rst) begin
        current_state <= IDLE;
        busy <= 1'b0;
    end
    else begin
        current_state <= next_state;
        busy <= busy_reg;
    end
end
// Next-state logic
always @(*) begin
    case (current_state)
        IDLE: begin
            ser_en = 0; mux_sel = 3'b000; busy_reg = 0;
            next_state = valid_data ? START : IDLE;
        end

        START: begin
            ser_en = 1; mux_sel = 3'b001; busy_reg = 1;
            next_state = DATA;
        end

        DATA: begin
            mux_sel = 3'b010; busy_reg = 1;
            if (ser_done) begin
                ser_en = 0;
                next_state = parity_en ? PARITY : STOP;
            end else begin
                ser_en = 1;
                next_state = DATA;
            end
        end

        PARITY: begin
            ser_en = 0; mux_sel = 3'b011; busy_reg = 1;
            next_state = STOP;
        end

        STOP: begin
            ser_en = 0; mux_sel = 3'b100; busy_reg = 1;
            next_state = IDLE;
        end

        default: begin
            ser_en = 0; mux_sel = 3'b000; busy_reg = 0;
            next_state = IDLE;
        end
    endcase
end
endmodule
```

3.3.5 uart tx.v – Top-Level Transmitter

The top-level module instantiates all four sub-modules and wires them together.

```
module uart_tx #(parameter DWIDTH = 8)
(clk, rst, p_data, data_valid, parity_en, parity_type,
s_data, busy);
input [DWIDTH-1:0] p_data;
input data_valid, parity_en, parity_type, clk, rst;
output s_data, busy;
wire parity_bit, ser_en, ser_done, ser_data;
wire [2:0] mux_sel;
// Parity block
parity_calc #(DWIDTH) par_c (
.clk(clk),
.rst(rst),
.data(p_data),
.parity_type(parity_type),
.parity_bit(parity_bit)
);
// Serializer
serializer #(DWIDTH) ser (
.clk(clk),
.rst(rst),
.p_data(p_data),
.ser_en(ser_en),
.s_data(ser_data),
.ser_done(ser_done)
);
// FSM
tx_fsm fsm (
.clk(clk),
.rst(rst),
.valid_data(data_valid),
.parity_en(parity_en),
.ser_done(ser_done),
.ser_en(ser_en),
.mux_sel(mux_sel),
.busy(busy)
);
// MUX
mux m (
.clk(clk),
.rst(rst),
.mux_sel(mux_sel),
.start_bit(1'b0),
.ser_data(ser_data),
.parity_bit(parity_bit),
.stop_bit(1'b1),
.tx_out(s_data)
);
endmodule
```

3.4 UART Receiver RTL Architecture

The UART receiver (uart rx.v) is built from seven modular sub-components, each responsible for a distinct stage of the reception pipeline.

Table 3.1: UART Receiver Sub-Module Descriptions

Module	Function
start_check.v	Detects the falling edge on s data; validates the start bit
edge_bit_counter.v	Generates timing signals based on 8× oversampling clock
data_sampling.v	Samples each data bit at the centre of the bit period (majority vote)
deserializer.v	Reconstructs 8-bit parallel data from incoming serial bits
parity_check.v	Computes expected parity and compares with received parity bit
stop_check.v	Validates that the stop bit is logic 1
rx_fsm.v	Controls all receiver states and output signal generation

3.4.1 data_sampling.v – Oversampled Bit Sampler

```
`timescale 1ns/1ps
module data_sampling #(parameter PWIDTH =6)(clk, rst, prescale, edge_counter,
data_sampling_en, rx_in, sampled_bit);
input clk, rst;
input [PWIDTH-1:0] prescale, edge_counter;
input data_sampling_en;
input rx_in;
output reg sampled_bit;
reg [2:0] samples;

always @(posedge clk or negedge rst) begin
    if(!rst) begin
        samples <= 3'b0;
        sampled_bit <= 1'b1; // Default to idle state (high)
    end
    else if(data_sampling_en) begin
        if(edge_counter == (prescale >> 1) - 1)
            samples[0] <= rx_in;
        else if(edge_counter == (prescale >> 1))
            samples[1] <= rx_in;
        else if(edge_counter == (prescale >> 1) + 1)
            samples[2] <= rx_in;
        else if(edge_counter == (prescale >> 1) + 2)
            sampled_bit <= (samples[0] & samples[1]) | (samples[1] & samples[2]) | (samples[0] &
samples[2]);
        end
        else begin
            samples <= 3'b0;
        end
    end
end
endmodule
```

3.4.2 rx_fsm.v – Receiver Finite State Machine

```
`timescale 1ns/1ps
module rx_fsm #(parameter PWIDTH =6)(clk, rst, prescale, edge_counter, bit_counter, rx_in, parity_en,
    stop_error, start_error, parity_error, data_sampling_en, deser_en, start_check_en, stop_check_en,
    parity_check_en, enable, data_valid);
input clk, rst;
input [PWIDTH-1:0] prescale, edge_counter;
input [PWIDTH-2:0] bit_counter;
input rx_in, parity_en;
input stop_error, start_error, parity_error;
output reg data_sampling_en, deser_en;
output reg start_check_en, stop_check_en, parity_check_en;
output reg enable, data_valid;
reg [2:0] current_state, next_state;
localparam IDLE =0, START =1, DATA =2, PARITY =3, STOP =4, CERROR =5, DVALID =6;

always @(posedge clk or negedge rst) begin
    if(!rst) current_state <= IDLE;
    else current_state <= next_state;
end
always @(*) begin
    case (current_state)
        IDLE: begin
            if(!rx_in) begin
                data_sampling_en = 1; enable = 1; deser_en = 0; data_valid = 0;
                stop_check_en = 0; start_check_en = 1; parity_check_en = 0;
                next_state = START;
            end else begin
                data_sampling_en = 0; enable = 0; deser_en = 0; data_valid = 0;
                stop_check_en = 0; start_check_en = 0; parity_check_en = 0;
                next_state = IDLE;
            end
        end
        START: begin
            data_sampling_en = 1; enable = 1; deser_en = 0; data_valid = 0;
            stop_check_en = 0; start_check_en = 1; parity_check_en = 0;
            if((bit_counter == 0) && (edge_counter == prescale-1)) begin
                if(!start_error) next_state = DATA;
                else next_state = IDLE;
            end else next_state = START;
        end
        DATA: begin
            data_sampling_en = 1; enable = 1; deser_en = 1; data_valid = 0;
            stop_check_en = 0; start_check_en = 0; parity_check_en = 0;
            if((bit_counter == 8) && (edge_counter == prescale-1)) begin
                if(parity_en) next_state = PARITY;
                else next_state = STOP;
            end else next_state = DATA;
        end
        PARITY: begin
            data_sampling_en = 1; enable = 1; deser_en = 0; data_valid = 0;
            stop_check_en = 0; start_check_en = 0; parity_check_en = 1;
            if((bit_counter == 9) && (edge_counter == prescale-1)) next_state = STOP;
            else next_state = PARITY;
        end
        STOP: begin
            data_sampling_en = 1; enable = 1; deser_en = 0; data_valid = 0;
            stop_check_en = 1; start_check_en = 0; parity_check_en = 0;
            if (parity_en) begin
                if ((bit_counter == 10) && (edge_counter == prescale - 1)) next_state = CERROR;
                else next_state = STOP;
            end else begin
                if ((bit_counter == 9) && (edge_counter == prescale - 1)) next_state = CERROR;
                else next_state = STOP;
            end
        end
        CERROR: begin
            data_sampling_en = 1; enable = 0; deser_en = 0; data_valid = 0;
            stop_check_en = 0; start_check_en = 0; parity_check_en = 0;
            if(parity_error || stop_error) next_state = IDLE;
            else next_state = DVALID;
        end
    endcase
end
```

```

DVALID: begin
    data_sampling_en = 0; enable = 0; deser_en = 0; data_valid = 1;
    stop_check_en = 0; start_check_en = 0; parity_check_en = 0;
    if(!rx_in) next_state = START;
    else      next_state = IDLE;
end
default: begin
    data_sampling_en = 0; enable = 0; deser_en = 0; data_valid = 0;
    stop_check_en = 0; start_check_en = 0; parity_check_en = 0;
    next_state = IDLE;
end
endcase
end
endmodule

```

3.4.3 uart rx.v – Top-Level Receiver

```

`timescale 1ns/1ps
module uart_rx #(parameter DWIDTH=8, PWIDTH=6)
    (clk, rst, s_data, parity_type, parity_en, prescale, p_data, data_valid);
    input clk, rst;
    input s_data, parity_type, parity_en;
    input [PWIDTH-1:0] prescale;
    output reg [DWIDTH-1:0] p_data;
    output data_valid;
    wire data_sampling_en, enable, deser_en, stop_check_en, start_check_en, parity_check_en;
    wire start_error, stop_error, parity_error;
    wire sampled_bit;
    wire [PWIDTH-1:0] edge_counter;
    wire [PWIDTH-2:0] bit_counter;
    wire [DWIDTH-1:0] internal_p_data;

    data_sampling #(PWIDTH) dsample(
        .clk(clk), .rst(rst), .prescale(prescale), .edge_counter(edge_counter),
        .data_sampling_en(data_sampling_en), .rx_in(s_data), .sampled_bit(sampled_bit)
    );
    deserializer #(DWIDTH, PWIDTH) deser(
        .clk(clk), .rst(rst), .deser_en(deser_en), .sampled_bit(sampled_bit),
        .prescale(prescale), .edge_counter(edge_counter), .p_data(internal_p_data)
    );
    edge_bit_counter #(PWIDTH) count(
        .clk(clk), .rst(rst), .enable(enable), .prescale(prescale),
        .bit_counter(bit_counter), .edge_counter(edge_counter)
    );
    rx_fsm #(PWIDTH) fsm(
        .clk(clk), .rst(rst), .prescale(prescale), .edge_counter(edge_counter),
        .bit_counter(bit_counter), .rx_in(s_data), .parity_en(parity_en),
        .stop_error(stop_error), .start_error(start_error), .parity_error(parity_error),
        .data_sampling_en(data_sampling_en), .deser_en(deser_en),
        .start_check_en(start_check_en), .stop_check_en(stop_check_en),
        .parity_check_en(parity_check_en), .enable(enable), .data_valid(data_valid)
    );
    start_check start(
        .clk(clk), .rst(rst), .sampled_bit(sampled_bit),
        .start_check_en(start_check_en), .start_error(start_error)
    );
    parity_check #(DWIDTH) parity(
        .clk(clk), .rst(rst), .parity_type(parity_type),
        .parity_check_en(parity_check_en), .sampled_bit(sampled_bit),
        .p_data(internal_p_data), .parity_error(parity_error)
    );
    stop_check stop(
        .clk(clk), .rst(rst), .stop_check_en(stop_check_en),
        .sampled_bit(sampled_bit), .stop_error(stop_error)
    );

    always @(posedge clk or negedge rst) begin
        if(!rst)
            p_data <= {DWIDTH{1'b0}};
        else if(data_valid)
            p_data <= internal_p_data;
        end
    end
endmodule

```

3.4.4.deserializer.v

```
`timescale 1ns/1ps
module deserializer #(parameter DWIDTH =8, PWIDTH =6)(clk, rst, deser_en, sampled_bit, prescale, edge_counter, p_data);
input clk, rst;
input deser_en, sampled_bit;
input [PWIDTH-1:0] prescale, edge_counter;
output reg [DWIDTH-1:0] p_data;
integer bit_index;
always @(posedge clk or negedge rst) begin
    if(!rst) begin
        p_data <= 0;
        bit_index <= 0;
    end
    else if (deser_en) begin
        if ((bit_index < DWIDTH) && (edge_counter == ((prescale >> 1) + 3))) begin
            p_data[bit_index] <= sampled_bit;
            bit_index <= bit_index + 1;
        end
    end
    else begin
        bit_index <= 0;
    end
end
endmodule
```

3.4.5. edge_bit_counter.v

```
`timescale 1ns/1ps
module edge_bit_counter #(parameter PWIDTH = 6)(clk, rst, enable, prescale, bit_counter, edge_counter);
input clk, rst, enable;
input [PWIDTH-1:0] prescale;
output reg [PWIDTH-2:0] bit_counter;
output reg [PWIDTH-1:0] edge_counter;
wire edge_counter_done;
assign edge_counter_done = (edge_counter == prescale-1) ? 1'b1 : 1'b0;
always @(posedge clk, negedge rst) begin
    if(!rst)
        edge_counter <= 0;
    else if(enable) begin
        if(!edge_counter_done)
            edge_counter <= edge_counter + 1;
        else
            edge_counter <= 0;
    end
    else
        edge_counter <= 0;
end
always @(posedge clk, negedge rst) begin
    if(!rst)
        bit_counter <= 0;
    else if(enable) begin
        if(edge_counter_done)
            bit_counter <= bit_counter + 1;
        else
            bit_counter <= bit_counter;
    end
    else
        bit_counter <= 0;
end
endmodule
```

3.4.6.parity.v

```
`timescale 1ns/1ps
module parity_check #(parameter DWIDTH =8)(clk, rst, parity_type, parity_check_en, sampled_bit,
p_data, parity_error);
input clk, rst;
input parity_type, parity_check_en, sampled_bit;
input [DWIDTH-1:0] p_data;
output reg parity_error;
reg parity_bit;
always @(posedge clk, negedge rst) begin
    if(!rst)
        parity_error <= 1'b0;
    else if (parity_check_en) begin
        if (parity_bit != sampled_bit)
            parity_error <= 1'b1;
        else
            parity_error <= 1'b0;
        end
    end
end
always @(*) begin
    if (parity_check_en) begin
        case(parity_type)
            1'b0: parity_bit = ^p_data;
            1'b1: parity_bit = ~^p_data;
            default: parity_bit = 1'b0;
        endcase
    end
    else parity_bit = 1'b0;
end
endmodule
```

3.4.7.start_bit.v

```
`timescale 1ns/1ps
module start_check (clk, rst, sampled_bit, start_check_en, start_error);
input clk, rst, sampled_bit;
input start_check_en;
output reg start_error;
always @(posedge clk, negedge rst) begin
    if(!rst)
        start_error <= 1'b0;
    else if (start_check_en)
        start_error <= sampled_bit;
end
endmodule
```

3.4.8. Stop_bit.v

```
`timescale 1ns/1ps
module stop_check (clk, rst, stop_check_en, sampled_bit, stop_error);
input clk, rst;
input stop_check_en;
input sampled_bit;
output reg stop_error;
always @(posedge clk, negedge rst) begin
    if(!rst)
        stop_error <= 1'b0;
    else if (stop_check_en) begin
        if (!sampled_bit)
            stop_error <= 1'b1;
        else
            stop_error <= 1'b0;
        end
    end
end
endmodule
```

3.5 SystemVerilog Testbench of Resiver

3.5.1 uart common.sv – Global Definitions

```
`timescale 1ns/1ps
`ifndef UART_COMMON_SV
`define UART_COMMON_SV
`define D_WIDTH 8
`define P_WIDTH 6
`define TIME_PERIOD 100
`define PRESCALE 8
class uart_common;
    static int count;
    static mailbox gen2bfm = new();
    static mailbox mon2sbd = new();
    static mailbox mon2cov = new();
    static string testname;
endclass
`endif
```

3.5.2 uart tx item.sv – Sequence Item (Transaction Object)

```
`timescale 1ns/1ps
`ifndef UART_TX_ITEM_SV
`define UART_TX_ITEM_SV
class uart_tx_item;
    bit unique_trans_data_q[$];
    rand bit [10:0] raw_data; // Using bit array for easier randomization
    bit s_data;
    rand bit parity_type;
    rand bit parity_en;
    rand bit [P_WIDTH-1:0] prescale;
    bit [D_WIDTH-1:0] p_data;
    bit valid_data;
    bit [D_WIDTH-1:0] actual_p_data;
    bit [D_WIDTH-1:0] expected_p_data;
    constraint start_stop_bit {
        soft raw_data[0] == 0; // Start bit
        soft (parity_en == 0) -> (raw_data[9] == 1); // Stop bit if no parity
        soft (parity_en == 1) -> (raw_data[10] == 1); // Stop bit if parity
    }
    constraint parity_generator {
        if (parity_en == 1) {
            (parity_type == 0) -> (raw_data[9] == (^raw_data[8:1])); // Even parity
            (parity_type == 1) -> (raw_data[9] == ~(^raw_data[8:1])); // Odd parity
        }
    }
    function void post_randomize();
        expected_p_data = raw_data[8:1];
        unique_trans_data_q.delete();
        unique_trans_data_q.push_back(raw_data[0]); // Start
        for (int i=1; i<=8; i++) unique_trans_data_q.push_back(raw_data[i]); // Data
        if (parity_en) unique_trans_data_q.push_back(raw_data[9]); // Parity
        unique_trans_data_q.push_back(parity_en ? raw_data[10] : raw_data[9]); // Stop
    endfunction

    function void print(input string name="tx");
        actual_p_data = {<<<p_data};
        $display("###%s### Valid=%0b Parity_en=%0s Parity_type=%0s Expected=%8b Received=%8b",
            name, valid_data, parity_en ? "YES" : "NO", parity_type ? "ODD" : "EVEN",
            expected_p_data, actual_p_data);
    endfunction
endclass
```

```

function void data_print(input string name="tx");
    $display("-----");
    $display("[%ot] %s: UART FRAME BREAKDOWN", $time, name);
    $write(" Serial Stream: [START:o] ");
    for(int i=1; i<=8; i++) $write("D%od:%b ", i-1, unique_trans_data_q[i]);
    if(parity_en) $write("[PARITY:%b] ", unique_trans_data_q[9]);
    $display("[STOP:1]");
    $display(" Expected Parallel Data: %8b", expected_p_data);
    if (name == "ScoreBoard") begin
        $display(" Received Parallel Data: %8b", actual_p_data);
        if (expected_p_data == actual_p_data)
            $display(" STATUS: MATCH!");
        else
            $display(" STATUS: MISMATCH!");
    end
    $display("-----");
endfunction
endclass
`endif

```

3.5.3 uart_intf.sv – SystemVerilog Interface

```

`ifndef UART_INTF_SV
`define UART_INTF_SV
interface uart_intf #(parameter DWIDTH=8, parameter PWIDTH=6)(input clk, rst);
    logic s_data = 1'b1;
    logic parity_type;
    logic parity_en;
    logic [PWIDTH-1:0] prescale;
    logic [DWIDTH-1:0] p_data;
    logic valid_data;
    bit pd_in[$];
    clocking bfm_cb @(posedge clk);
        default input #0 output #1;
        input valid_data;
        input p_data;
        output prescale;
        output parity_en;
        output parity_type;
        output s_data;
    endclocking
    clocking mon_cb @(posedge clk);
        default input #0 output #1;
        input valid_data;
        input p_data;
        input prescale;
        input parity_en;
        input parity_type;
        input s_data;
    endclocking
endinterface
`endif

```

3.5.4 uart_gen.sv – Generator

The generator runs 4 iterations per test phase, randomizing the sequence item with constraints based on the active testname.

```
`timescale 1ns/1ps
`ifndef UART_GEN_SV
`define UART_GEN_SV
class uart_gen;
  task run();
    $display("[%0t] Generator: Starting", $time);

    // Let system reset first
    #200;

    // High toggling patterns to verify s_data transitions
    send_frame(8'hAA); // 10101010
    #(`PRESCALE * 8 * 10);

    send_frame(8'h55); // 01010101
    #(`PRESCALE * 8 * 10);

    // Additional diverse patterns
    send_frame(8'hCC); // 11001100
    #(`PRESCALE * 8 * 10);

    send_frame(8'h33); // 00110011
    #(`PRESCALE * 8 * 10);

    // Some standard frames
    send_frame(8'h5D);
    #(`PRESCALE * 8 * 10);

    send_frame(8'h0F);
    #(`PRESCALE * 8 * 10);

    send_frame(8'hD7);
    #(`PRESCALE * 8 * 10);

    #1000;
  endtask

  task send_frame(bit [7:0] data_val);
    uart_tx_item tx = new();
    $display("[%0t] Generator: Creating transaction for %h", $time, data_val);

    assert(tx.randomize() with {
      tx.raw_data[8:1] == data_val;
      tx.parity_en == 0;
      tx.parity_type == 0;
      tx.prescale == `PRESCALE;
      tx.raw_data[0] == 0; // Start bit
      tx.raw_data[9] == 1; // Stop bit
    });

    uart_common::gen2bfm.put(tx);
    $display("[%0t] Generator: Transaction put in mailbox", $time);
  endtask
endclass
`endif
```

3.5.5 uart_bfm.sv – BFM Driver

```
`timescale 1ns/1ps
`ifndef UART_BFM_SV
`define UART_BFM_SV
class uart_bfm;
    uart_tx_item tx;
    virtual uart_intf #(`D_WIDTH, `P_WIDTH) vif;
    function new();
        vif = top.pif;
    endfunction
    task run();
        $display("[%0t] BFM: Starting", $time);
        forever begin
            uart_common::gen2bfm.get(tx);
            $display("[%0t] BFM: Got transaction, driving...", $time);
            uart_drive(tx);
        end
    endtask
    task uart_drive(uart_tx_item tx);
        // Set the configuration inputs before driving the frame
        vif.bfm_cb.parity_type <= tx.parity_type;
        vif.bfm_cb.parity_en  <= tx.parity_en;
        vif.bfm_cb.prescale   <= tx.prescale;
        vif.pd_in             = tx.unique_trans_data_q;

        // Drive IDLE for some cycles before frame
        repeat(50) @(vif.bfm_cb) vif.bfm_cb.s_data <= 1'b1;

        foreach(tx.unique_trans_data_q[i]) begin
            @(vif.bfm_cb) begin
                vif.bfm_cb.s_data <= tx.unique_trans_data_q[i];
            end
            repeat(`PRESCALE-1) @(vif.bfm_cb);
        end

        // Drive IDLE for some cycles after frame
        repeat(50) @(vif.bfm_cb) vif.bfm_cb.s_data <= 1'b1;
    endtask
endclass
`endif
```

3.5.6 uart_mon.sv – Monitor

```
`timescale 1ns/1ps
`ifndef UART_MON_SV
`define UART_MON_SV
class uart_mon;
    uart_tx_item tx;
    virtual uart_intf #(`D_WIDTH, `P_WIDTH) vif;
    function new();
        vif = top.pif;
    endfunction
    task run();
        $display("[%0t] Monitor: Starting", $time);
        forever begin
            @(vif.mon_cb);
            if (vif.mon_cb.valid_data == 1'b1) begin
                tx = new();
                tx.s_data = vif.mon_cb.s_data;
                tx.prescale = vif.mon_cb.prescale;
                tx.parity_en = vif.mon_cb.parity_en;
                tx.parity_type = vif.mon_cb.parity_type;
                tx.unique_trans_data_q = vif.pd_in;
                tx.p_data = vif.mon_cb.p_data;
                tx.valid_data = vif.mon_cb.valid_data;

                // Extract expected parallel data from the serial frame
                for(int i=0; i<`D_WIDTH; i++) begin
                    tx.expected_p_data[i] = tx.unique_trans_data_q[i+1];
                end

                uart_common::mon2sbd.put(tx);
                uart_common::mon2cov.put(tx);
            end
        end
    endtask
endclass
`endif
```

3.5.7 uart sbd.sv – Scoreboard (Self-Checking)

The scoreboard automatically verifies every transmitted UART frame. It reconstructs the frame from the sampled serial bits and checks: start bit = 0, data bits match p data, parity bit matches expected value, and stop bit = 1.

```
`timescale 1ns/1ps
`ifndef UART_SBD_SV
`define UART_SBD_SV
class uart_sbd;
    uart_tx_item tx;
    task run();
        $display("[%0t] Scoreboard: Starting", $time);
        forever begin
            uart_common::mon2sbd.get(tx);
            $display("[%0t] Scoreboard: Got transaction", $time);
            if(tx.valid_data==1) begin
                if(tx.expected_p_data == tx.p_data) begin
                    tx.actual_p_data = tx.p_data; // Keep it consistent for printing if needed
                    tx.data_print("ScoreBoard");
                    $display("[%0t] SCOREBOARD: MATCH! Sent:%8b Received:%8b", $time,
tx.expected_p_data, tx.actual_p_data);
                    $display("Success: Received Data correctly from testname : %0s\n",
uart_common::testname);
                end else begin
                    tx.actual_p_data = tx.p_data;
                    tx.data_print("ScoreBoard");
                    $display("[%0t] SCOREBOARD: MISMATCH! Sent:%8b Received:%8b", $time,
tx.expected_p_data, tx.actual_p_data);
                    $display("Error: Received Data wrong from testname : %0s\n",
uart_common::testname);
                end
            end
        end
    endtask
endclass
`endif
```

3.5.8 uart cov.sv – Functional Coverage

```
`ifndef UART_COV_SV
`define UART_COV_SV
class uart_cov;
    uart_tx_item tx;
    covergroup uart_cg;
        cov_valid_data : coverpoint tx.valid_data {
            bins valid_data_1_o[] = (1'b1=>1'b0);
            bins valid_data_o_1[] = (1'b0=>1'b1);
        }
        cov_s_data : coverpoint tx.s_data {
            bins s_data_1_o[] = (1'b1=>1'b0);
            bins s_data_o_1[] = (1'b0=>1'b1);
        }
        cov_parity_en : coverpoint tx.parity_en {
            bins parity_en_yes[] = {1'b1};
            bins parity_en_no[] = {1'b0};
        }
        cov_parity_type : coverpoint tx.parity_type {
            bins parity_type_odd[] = {1'b1};
            bins parity_type_even[] = {1'b0};
        }
    endgroup
    function new();
        uart_cg = new();
    endfunction
    task run();
        forever begin
            uart_common::mon2cov.get(tx);
            uart_cg.sample();
        end
    endtask
endclass
`endif
```

3.5.9. uart agent.sv and uart_env.sv

```
`ifndef UART_AGENT_SV
`define UART_AGENT_SV
class uart_agent;
    uart_gen gen;
    uart_bfm bfm;
    uart_mon mon;
    uart_cov cov;
    task run();
        gen = new(); bfm = new(); mon = new(); cov = new();
        fork gen.run(); bfm.run(); mon.run(); cov.run(); join
    endtask
endclass
`endif
```

3.5.10. uart env.sv

```
`ifndef UART_ENV_SV
`define UART_ENV_SV
class uart_env;
    uart_agent agent;
    uart_sbd sbd;
    task run();
        agent = new(); sbd = new();
        fork agent.run(); sbd.run(); join
    endtask
endclass
`endif
```

3.5.11. uart top.sv – Top-Level Test Module

```
`timescale 1ns/1ps
module top;
    bit clk, rst;
    uart_env env;
    event e;

    uart_rx #(`D_WIDTH, `P_WIDTH) dut (
        .clk(pif.clk),
        .rst(pif.rst),
        .s_data(pif.s_data),
        .parity_type(pif.parity_type),
        .parity_en(pif.parity_en),
        .prescale(pif.prescale),
        .p_data(pif.p_data),
        .data_valid(pif.valid_data)
    );
    uart_intf #(`D_WIDTH, `P_WIDTH) pif(clk, rst);

    initial begin clk=0; forever #(`TIME_PERIOD/2) clk=~clk; end
    initial begin
        $display("[%0t] Top: Reset starting", $time);
        rst=0; repeat(2) @(posedge clk);
        rst=1;
        $display("[%0t] Top: Reset released, starting environment", $time);
        env=new();
        env.run();
    end
    initial begin
        #(3000 * `TIME_PERIOD); // wait enough time for all 7 frames to process (each
        takes ~200 clocks)
        $finish;
    end
endmodule
```

3.5.12. list.sv

```
`timescale 1ns/1ps
`ifndef LIST_SV
`define LIST_SV
`include "uart_common.sv"
`include "uart_tx_item.sv"
`include "uart_sbd.sv"
`include "uart_cov.sv"
`include "uart_mon.sv"
`include "uart_bfm.sv"
`include "uart_gen.sv"
`include "uart_agent.sv"
`include "uart_env.sv"
`endif
```

3.5.13. Verilog Testbench(uart_rx_tb.v)

```
`define TIMEPERIOD 8
module uart_rx_tb;
reg clk, rst, s_data, parity_type, parity_en;
reg [5:0] prescale;
wire [7:0] p_data;
wire data_valid;

uart_rx #(8, 6) uut (
    .clk(clk), .rst(rst), .s_data(s_data), .parity_type(parity_type),
    .parity_en(parity_en), .prescale(prescale), .p_data(p_data), .data_valid(data_valid)
);

initial begin clk = 0; forever #(`TIMEPERIOD/2) clk = ~clk; end

initial begin
    rst=0; s_data=1; parity_type=0; parity_en=0; prescale=8;
    #10 rst=1;
    #(prescale*2);
    parity_en=0; parity_type=0; send_uart_frame(8'b10101010, 0, 0, 1);
    #(prescale*20);
    parity_en=0; parity_type=1; send_uart_frame(8'b01010101, 0, 1, 1);
    #(prescale*20);
    parity_en=1; parity_type=1; send_uart_frame(8'b10101010, 1, 1, 1);
    #(prescale*20);
    parity_en=1; parity_type=0; send_uart_frame(8'b01010101, 1, 0, 1);
    #(prescale*30);
    $finish;
end

task send_uart_frame;
input [7:0] data;
input parity_en_t;
input parity_type_t;
input stop_bit;
integer i;
begin
    s_data = 0; #(prescale * `TIMEPERIOD);
    for (i = 0; i < 8; i = i+1) begin
        s_data = data[i]; #(prescale * `TIMEPERIOD);
    end
    if (parity_en_t) begin
        if (parity_type_t) s_data = ~^(data);
        else s_data = ^(data);
        #(prescale * `TIMEPERIOD);
    end
    s_data = stop_bit; #(prescale * `TIMEPERIOD);
end
endtask

initial begin $dumpfile("uart_rx_tb.vcd"); $dumpvars; end
endmodule
```

3.6 Transmitter Testbench:

3.6.1 uart common.sv – Global Definitions

```
`define D_WIDTH 8
`define TIME_PERIOD 8
class uart_common;
    static mailbox gen2bfm = new();
    static mailbox mon2sbd = new();
    static mailbox mon2cov = new();
    static string testname;
endclass
```

3.6.2 uart tx item.sv – Sequence Item (Transaction Object)

```
class uart_tx_item;
    bit valid_data;
    rand bit parity_en;
    rand bit parity_type;
    rand bit [`D_WIDTH-1:0] p_data;
    bit s_data;
    bit busy;
    function void print(input string name="tx");
        $display("The Print Statement is from ###%s###", name);
        $display("Valid_data=%0b", valid_data);
        $display("Parity_enabled=%0s", parity_en ? "YES" : "NO");
        $display("Parity_type=%0s", parity_type ? "ODD" : "EVEN");
        $display("Given_Parallel_data=%8b", p_data);
        $display("Serial_Data=%0b", s_data);
        $display("Busy=%0b", busy);
    endfunction
endclass
```

3.6.3 uart_intf.sv – SystemVerilog Interface

```
interface uart_intf #(parameter DWIDTH = 8)(input clk, rst);
    logic valid_data;
    logic parity_en;
    logic parity_type;
    logic [`D_WIDTH-1:0] p_data;
    logic s_data;
    logic busy;
    clocking bfm_cb @(posedge clk);
        default input #0 output #1;
        output valid_data;
        output parity_en;
        output parity_type;
        output p_data;
        input s_data;
        input busy;
    endclocking
    clocking mon_cb @(posedge clk);
        default input #0 output #1;
        input valid_data;
        input parity_en;
        input parity_type;
        input p_data;
        input s_data;
        input busy;
    endclocking
endinterface
```

3.6.4 uart_gen.sv – Generator

```
class uart_gen;
    task run();
        repeat (4) begin
            uart_tx_item tx = new();
            case(uart_common::testname)
                "no_parity1": begin
                    assert(tx.randomize() with {tx.parity_en==0; tx.parity_type==0;});
                    uart_common::gen2bfm.put(tx);
                end
                "no_parity2": begin
                    assert(tx.randomize() with {tx.parity_en==0; tx.parity_type==1;});
                    uart_common::gen2bfm.put(tx);
                end
                "even_parity": begin
                    assert(tx.randomize() with {tx.parity_en==1; tx.parity_type==0;});
                    uart_common::gen2bfm.put(tx);
                end
                "odd_parity": begin
                    assert(tx.randomize() with {tx.parity_en==1; tx.parity_type==1;});
                    uart_common::gen2bfm.put(tx);
                end
            endcase
            #100;
            wait((top.pif.bfm_cb.busy==0) && (top.pif.bfm_cb.valid_data==0));
        end
    endtask
endclass
```

3.6.5 uart_bfm.sv – BFM Driver

```
class uart_bfm;
  uart_tx_item tx;
  virtual uart_intf #( `D_WIDTH) vif;
  function new();
    vif = top.pif;
  endfunction
  task run();
    forever begin
      uart_common::gen2bfm.get(tx);
      uart_drive(tx);
    end
  endtask
  task uart_drive(uart_tx_item tx);
    repeat(2) begin
      @(vif.bfm_cb);
      vif.bfm_cb.p_data    <= tx.p_data;
      vif.bfm_cb.parity_en <= tx.parity_en;
      vif.bfm_cb.parity_type <= tx.parity_type;
    end
    @(vif.bfm_cb);
    vif.bfm_cb.valid_data <= 1'b1;
    @(vif.bfm_cb) begin
      tx.s_data = vif.bfm_cb.s_data;
      tx.busy   = vif.bfm_cb.busy;
    end
    wait(vif.bfm_cb.busy == 1);
    @(vif.bfm_cb);
    vif.bfm_cb.valid_data <= 1'b0;
  endtask
endclass
```

3.6.6 uart_mon.sv – Monitor

```
class uart_mon;
  uart_tx_item tx;
  virtual uart_intf #( `D_WIDTH) vif;
  function new();
    vif = top.pif;
  endfunction
  task run();
    forever begin
      @(vif.mon_cb) begin
        tx = new();
        tx.valid_data = vif.mon_cb.valid_data;
        tx.parity_en  = vif.mon_cb.parity_en;
        tx.parity_type = vif.mon_cb.parity_type;
        tx.p_data     = vif.mon_cb.p_data;
        tx.s_data     = vif.mon_cb.s_data;
        tx.busy       = vif.mon_cb.busy;
      end
      uart_common::mon2sbd.put(tx);
      uart_common::mon2cov.put(tx);
    end
  endtask
endclass
```

3.6.7 uart sbd.sv – Scoreboard (Self-Checking)

The scoreboard automatically verifies every transmitted UART frame. It reconstructs the frame from the sampled serial bits and checks: start bit = 0, data bits match p data, parity bit matches expected value, and stop bit = 1.

```
class uart_sbd;
  uart_tx_item tx;
  bit start_bit, parity_bit, parity_bit_expected, stop_bit;
  bit data_q[`D_WIDTH];
  bit [`D_WIDTH-1:0] data;
  uart_tx_item data_queue[$];
  bit s_data_queue[$:`D_WIDTH+3];

  task run();
    forever begin
      uart_common::mon2sbd.get(tx);
      if (tx.busy) begin
        data_queue.push_front(tx);
      end
      if (data_queue.size() != 0) begin
        tx = data_queue.pop_back();
        s_data_queue.push_front(tx.s_data);
        if (tx.parity_en) begin
          if (s_data_queue.size() == `D_WIDTH+3) begin
            start_bit = s_data_queue.pop_back();
            foreach (data_q[i]) data_q[i] = s_data_queue.pop_back();
            parity_bit = s_data_queue.pop_back();
            stop_bit = s_data_queue.pop_back();
            data = {>>{data_q}};
            $display("start = %0b", start_bit);
            $display("data = %8b", data);
            $display("parity= %0b", parity_bit);
            $display("stop = %0b", stop_bit);
            if (start_bit == 0) $display("PASSED START: start bit = %0b", start_bit);
            else $error ("FAILED START: start bit = %0b", start_bit);
            if (tx.p_data == data) $display("PASSED DATA: data = %8b, p_data = %8b", data, tx.p_data);
            else $error ("FAILED DATA: data = %8b, p_data = %8b", data, tx.p_data);
            parity_bit_expected = (tx.parity_type == 0) ? ^tx.p_data : ~(^tx.p_data);
            if (parity_bit == parity_bit_expected) $display("PASSED PARITY: parity = %0b, expected = %0b", parity_bit,
            parity_bit_expected);
            else $error ("FAILED PARITY: parity = %0b, expected = %0b", parity_bit,
            parity_bit_expected);
            if (stop_bit == 1) $display("PASSED STOP: stop bit = %0b", stop_bit);
            else $error ("FAILED STOP: stop bit = %0b", stop_bit);
          end
        end else begin
          if (s_data_queue.size() == `D_WIDTH+2) begin
            start_bit = s_data_queue.pop_back();
            foreach (data_q[i]) data_q[i] = s_data_queue.pop_back();
            stop_bit = s_data_queue.pop_back();
            data = {>>{data_q}};
            $display("start = %0b", start_bit);
            $display("data = %8b", data);
            $display("stop = %0b", stop_bit);
            if (start_bit == 0) $display("PASSED START: start bit = %0b", start_bit);
            else $error ("FAILED START: start bit = %0b", start_bit);
            if (tx.p_data == data) $display("PASSED DATA: data = %8b, p_data = %8b", data, tx.p_data);
            else $error ("FAILED DATA: data = %8b, p_data = %8b", data, tx.p_data);
            if (stop_bit == 1) $display("PASSED STOP: stop bit = %0b", stop_bit);
            else $error ("FAILED STOP: stop bit = %0b", stop_bit);
          end
        end
      end
    end
  endtask
endclass
```

3.6.8 uart cov.sv – Functional Coverage

```
class uart_cov;
  uart_tx_item tx;
  covergroup cg;
    cov_valid_data : coverpoint tx.valid_data {
      bins valid_data_1_o[] = (1'b1=>1'b0);
      bins valid_data_o_1[] = (1'b0=>1'b1);
    }
    cov_busy_check : coverpoint tx.busy {
      bins busy_1_o[] = (1'b1=>1'b0);
      bins busy_o_1[] = (1'b0=>1'b1);
    }
    cov_parity_en : coverpoint tx.parity_en {
      bins parity_en_yes[] = {1'b1};
      bins parity_en_no[] = {1'b0};
    }
    cov_parity_type : coverpoint tx.parity_type {
      bins parity_type_odd[] = {1'b1};
      bins parity_type_even[] = {1'b0};
    }
  endgroup
  function new();
    cg = new();
  endfunction
  task run();
    forever begin
      uart_common::mon2cov.get(tx);
      cg.sample();
    end
  endtask
endclass
```

3.5.9. uart agent.sv and uart_env.sv

```
class uart_agent;
  uart_gen gen;
  uart_bfm bfm;
  uart_mon mon;
  uart_cov cov;
  task run();
    gen = new();
    bfm = new();
    mon = new();
    cov = new();
    fork
      gen.run();
      bfm.run();
      mon.run();
      cov.run();
    join
  endtask
endclass
```

3.6.10. uart env.sv

```
class uart_env;
    uart_agent agent;
    uart_sbd sbd;
    task run();
        agent = new();
        sbd = new();
        fork
            agent.run();
            sbd.run();
        join
    endtask
endclass
```

3.6.11. uart top.sv – Top-Level Test Module

```
module top;
    bit clk;
    bit rst;
    uart_env env;
    event e;

    uart_tx #(D_WIDTH) dut (
        .clk(clk),
        .rst(rst),
        .p_data(pif.p_data),
        .data_valid(pif.valid_data),
        .parity_en(pif.parity_en),
        .parity_type(pif.parity_type),
        .s_data(pif.s_data),
        .busy(pif.busy)
    );

    uart_intf pif(clk, rst);

    initial begin
        clk = 1'b0;
        forever #(`TIME_PERIOD/2) clk = ~clk;
    end

    initial begin
        rst = 1'b0;
        repeat(2) @(posedge clk);
        rst = 1'b1;
        ->e;
    end

    initial begin
        wait(e.triggered);
        env = new();
        env.run();
    end

    initial begin
        uart_common::testname = "no_parity1"; #140;
        uart_common::testname = "no_parity2"; #140;
        uart_common::testname = "even_parity"; #140;
        uart_common::testname = "odd_parity"; #140;
        $finish;
    end
endmodule
```

3.6.12. list.sv

```
`include "uart_tx.v"
`include "uart_common.sv"
`include "uart_intf.sv"
`include "uart_tx_item.sv"
`include "uart_sbd.sv"
`include "uart_cov.sv"
`include "uart_mon.sv"
`include "uart_bfm.sv"
`include "uart_gen.sv"
`include "uart_agent.sv"
`include "uart_env.sv"
`include "uart_top.sv"
```

3.6.13. Transmitter Verilog Testbench Code

```
module uart_tx_tb;
// Testbench signals
reg clk;
reg rst;
reg [7:0] p_data;
reg data_valid;
reg parity_en;
reg parity_type;
wire s_data;
wire busy;
// Instantiate UART transmitter
uart_tx #(8) uut (
    .clk(clk),
    .rst(rst),
    .p_data(p_data),
    .data_valid(data_valid),
    .parity_en(parity_en),
    .parity_type(parity_type),
    .s_data(s_data),
    .busy(busy)
);
// Clock generation
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
// Test sequence
initial begin
    // Initialize inputs
    rst = 0;
    p_data = 8'b00000000;
    data_valid = 0;
    parity_en = 0;
    parity_type = 0;
    // Apply reset
    #10 rst = 1;
    // Test case 1: Transmit data with no parity
    #20;
    p_data = 8'b01010101;
    data_valid = 1;
    parity_en = 0;
    parity_type = 0;
    #150;
    p_data = 8'b10101010;
    data_valid = 1;
    parity_en = 0;
    parity_type = 1;
end
```

```

#150;
p_data  = 8'b01010101;
data_valid = 1;
parity_en = 1;
parity_type = 0;
#150;
p_data  = 8'b10101010;
data_valid = 1;
parity_en = 1;
parity_type = 1;
#150;
$finish;
end
// Display the values
initial begin
    $monitor("p_data : %0b , parity_enable : %0b , s_data : %0b , busy : %0b ",
        p_data, parity_en, s_data, busy);
end
initial begin
    $dumpfile("uart_tx_tb.vcd");
    $dumpvars;
end
endmodule

```


4 Results

4.1 UART Transmitter Simulation Waveform

The UART transmitter was simulated using ModelSim with the SystemVerilog testbench. Key observations from the waveform:

- The valid data signal goes high for one clock cycle to latch parallel input.
- The busy signal asserts immediately and remains high throughout the entire frame.
- The s_data output carries the serialized frame: start bit (0), 8 data bits LSB-first, optional parity bit, stop bit (1).
- The p_data bus shows values 0x5d, 0x78, and 0x8f transmitted in successive frames.

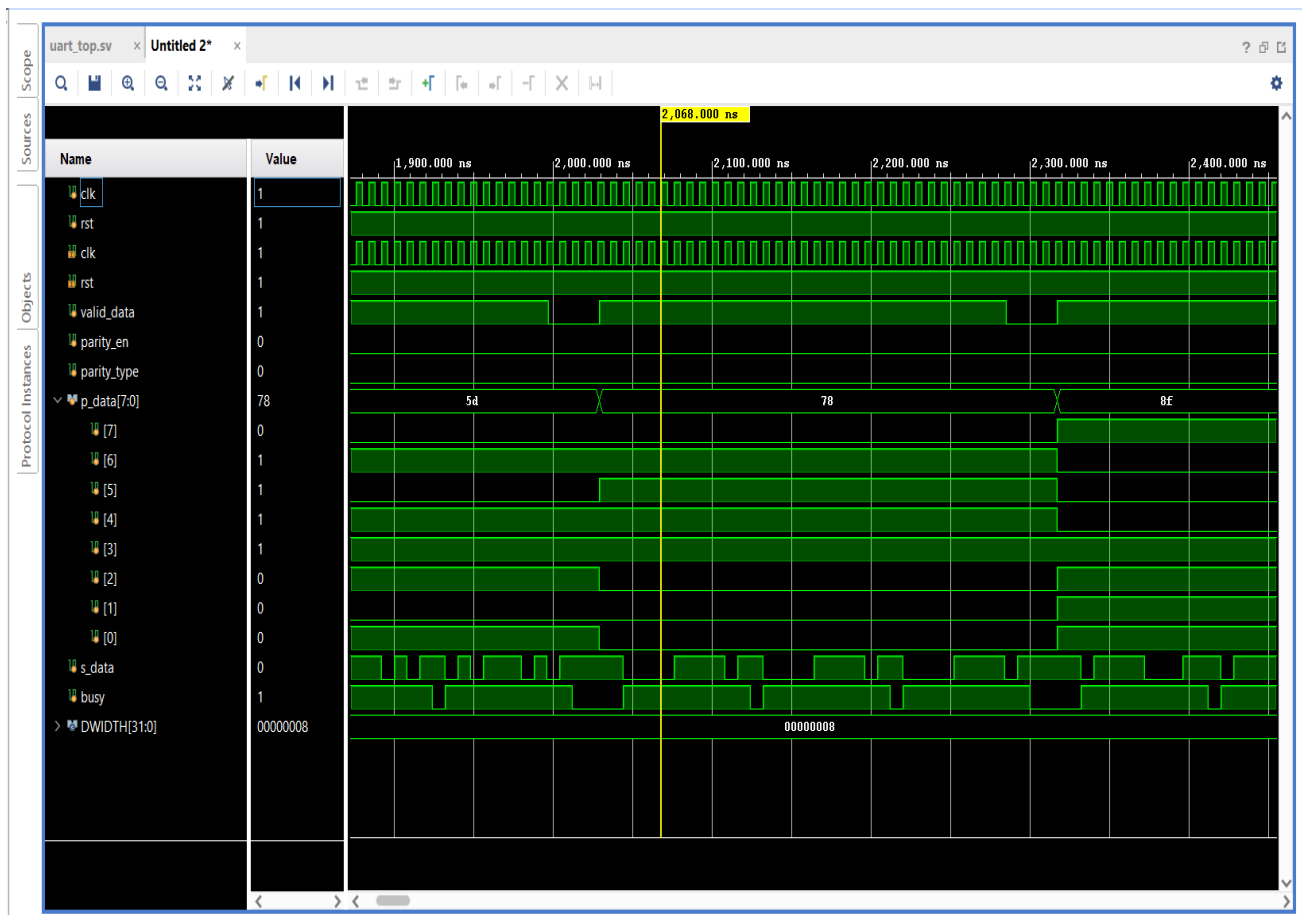


Figure 4.1: UART Transmitter Simulation Waveform (ModelSim)

UART Transmitter Simulation Explanation

The simulation waveform shows the working of a UART transmitter, which converts parallel data into serial form. The **clk** signal is continuously running to control the operation, and the **rst** signal initializes the system at the beginning. When **valid_data** becomes high, the transmitter starts sending the input data **p_data = 78 (01001110 in binary)**. This data is loaded and prepared for transmission in serial format.

The UART frame in this simulation includes a start bit, 8 data bits, and a stop bit, with parity disabled (**parity_en = 0**). First, a **start bit (0)** is sent, followed by the **8 data bits** transmitted one by one (LSB first). Since parity is disabled, no parity bit is included. Finally, a **stop bit (1)** is sent, which brings the line back to idle state. The **busy** signal stays high during transmission and becomes low after completion, confirming that the UART transmitter is working correctly.

4.2. Schematic Diagram of Transmitter:

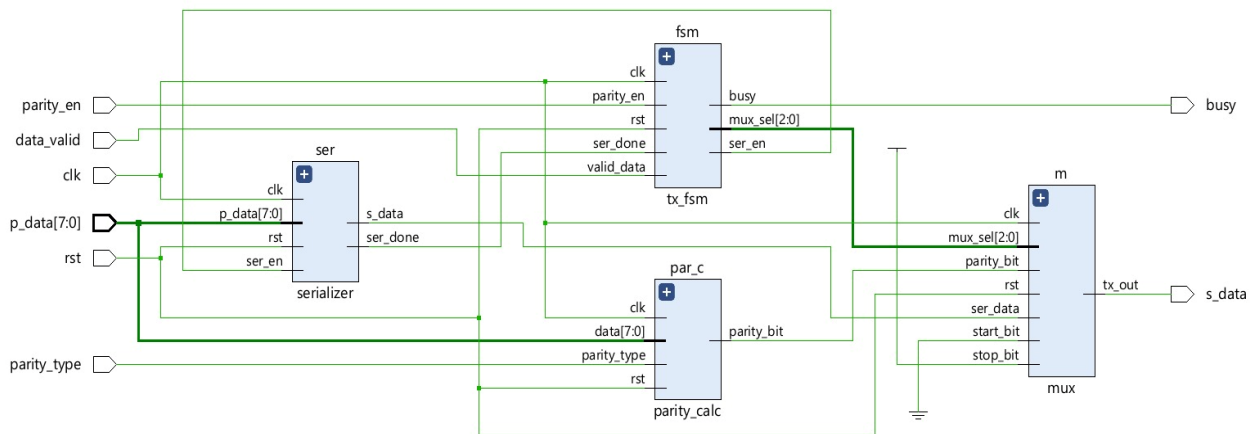


Figure 4.2. Schematic Diagram of Transmitter.

4.3. UART Receiver Simulation Waveform

The receiver simulation waveform confirms correct 8× oversampled data reception:

- The **s_data** input carries the incoming serial frame.
- The **p_data** output reconstructs values 0x00, 0xd7, 0xf8, 0xfd after complete frame reception.
- The data valid output asserts for exactly one clock cycle per frame.
- **prescale[5:0] = 08** confirms 8× oversampling timing.
- Back-to-back frames are correctly handled with no framing errors.

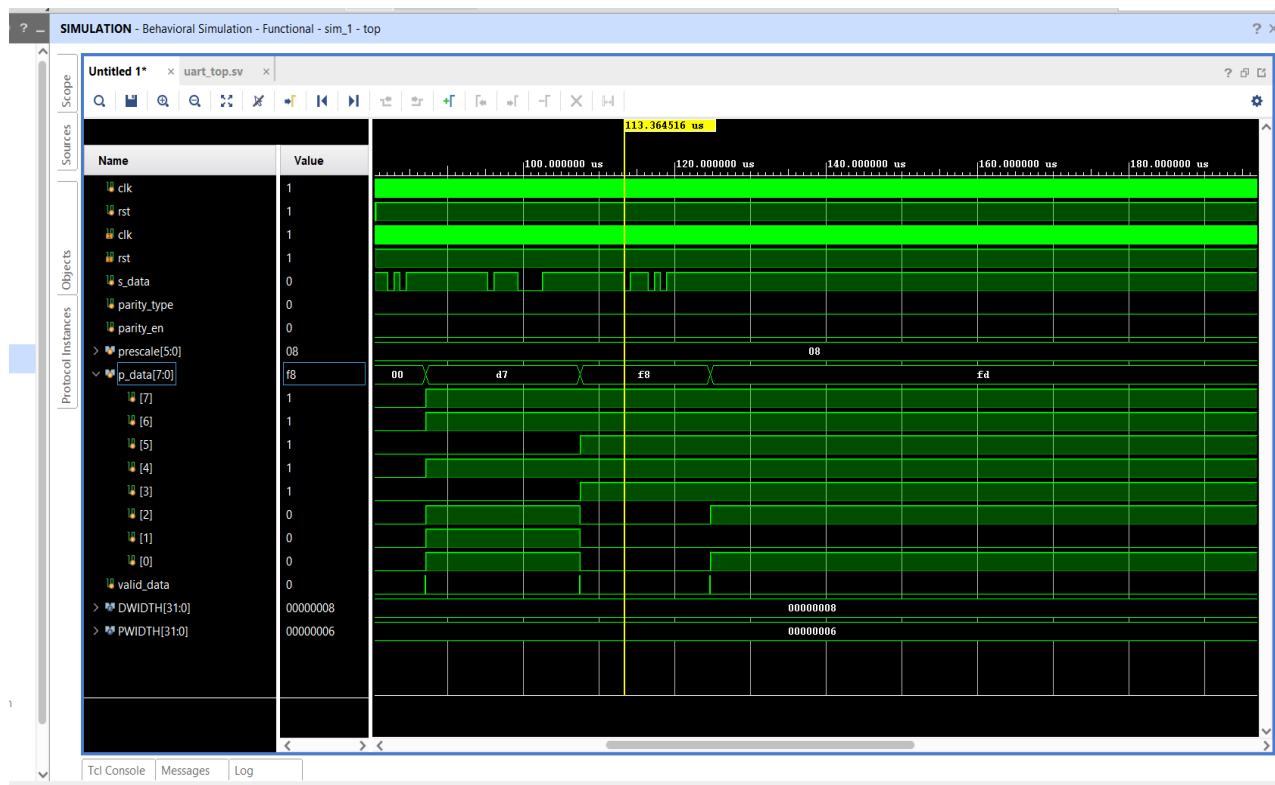


Figure 4.3: UART Receiver Simulation Waveform

UART Receiver Simulation Explanation:

The simulation shows the working of a UART receiver, which converts serial data into parallel form. The **clk** signal is continuously running to control the operation, and the **rst** signal initializes the system at the beginning. The **s_data** line is initially at logic 1, which represents the idle state. When the receiver detects a transition from 1 to 0, it identifies the **start bit**, indicating that data transmission has begun. The sampling of incoming bits is controlled by the **prescale value (08)**, which sets the baud rate timing.

After detecting the start bit, the receiver reads the incoming bits one by one and reconstructs the data. The received data is **F8 (11111000 in binary)**, which matches the waveform. Since **parity_en = 0**, parity checking is disabled, so no parity bit is considered. After all 8 bits are received, the receiver checks the **stop bit (1)**, confirming the end of the frame. Finally, the output **p_data** becomes **F8**, and **valid_data** goes **high**, indicating successful data reception.

4.4. Schematic Diagram Of Reciver:

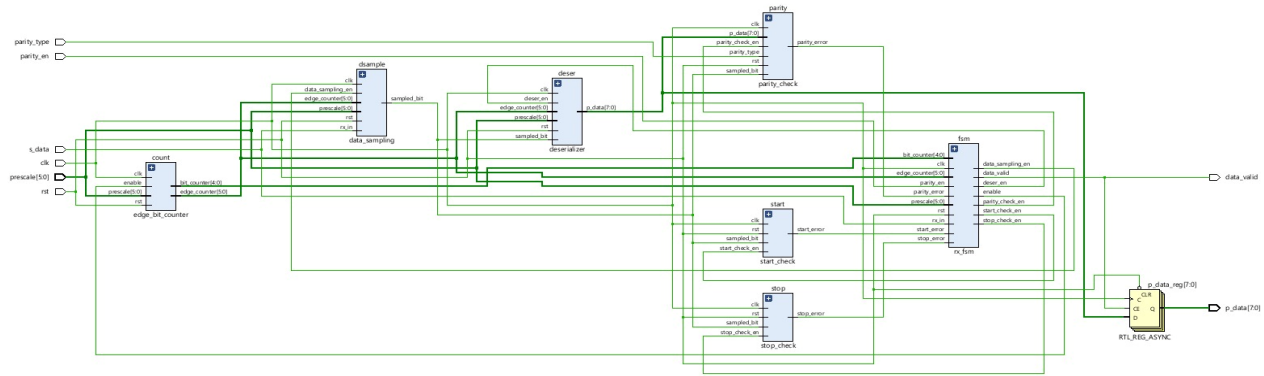


Figure 4.4. Schematic Diagram of Uart Reciver.

4.5.UVM Scoreboard Console Output of Transmitter.

The UVM simulation log confirms that all four test scenarios passed without errors. The scoreboard verified each frame's start bit, data bits, parity bit (where applicable), and stop bit individually.

```
# --- Test: no_parity ---
# data = 01110010
# stop = 1
# PASSED START : The start bit = 0
# PASSED DATA : The data = 01110010 , p_data = 01110010
# PASSED STOP : The stop bit = 1
# --- Test: even_parity ---
# start = 0
# data = 00000101
# parity = 0
# stop = 1
# PASSED START : The start bit = 0
# PASSED DATA : The data = 00000101 , p_data = 00000101
# PASSED PARITY : The parity bit = 0 , Expected parity bit = 0
# PASSED STOP : The stop bit = 1
# --- Test: odd_parity ---
# start = 0
# data = 00000101
# parity = 1
# stop = 1
# PASSED START : The start bit = 0
# PASSED DATA : The data = 00000101 , p_data = 00000101
# PASSED PARITY : The parity bit = 1 , Expected parity bit = 1
# PASSED STOP : The stop bit = 1
```

4.6.summary

Test Case	Parity Configuration	Transactions	Result	Errors
no parity1	parity_en = 0, type = 0	4	PASSED	0
no parity2	parity_en = 0, type = 1	4	PASSED	0
even parity	parity_en = 1, type = 0 (EVEN)	4	PASSED	0
odd parity	parity_en = 1, type = 1 (ODD)	4	PASSED	0
Total	–	16		

5 Conclusion

This project successfully demonstrates the complete RTL design and SystemVerilog-based functional verification of a UART communication protocol.

Transmitter: The UART transmitter was designed using a clean hierarchical Verilog architecture comprising five modules — mux, parity calc, serializer, tx fsm, and uart tx — each performing a well-defined function. The design supports configurable parity (even/odd/none), a 200 MHz operating clock, and generates all UART frame formats correctly.

Receiver: The UART receiver employs an 8× oversampling technique with 3-sample majority-vote logic for robust bit sampling, and includes parity error detection and stop-bit error detection to identify corrupted frames before forwarding valid data downstream. The receiver correctly handles consecutive frames with no data loss across all simulated test scenarios.

Verification: A structured SystemVerilog testbench was built following industry-standard verification practices, including constrained-random stimulus generation, self-checking scoreboard, and functional coverage measurement. All four test scenarios — no parity1, no_parity2, even parity, and odd parity — passed with zero errors across all frame fields.

Skills Acquired:

- Digital RTL design using Verilog HDL with modular, parameterizable architecture.
- Finite State Machine (FSM) design and implementation for both TX and RX.
- Systematic functional verification using SystemVerilog testbenches.
- Constrained-random verification and functional coverage measurement.
- Simulation and waveform analysis using ModelSim/Vivado.
- The complete design is parameterizable, synthesizable, and directly applicable to real-world FPGA and ASIC integration projects.

6. Bibliography

- [1] R, Y. & V, R. M. (2020). *Design and Verification of UART using System Verilog*. International Journal of Engineering and Advanced Technology, 9(5), 1208–1211.
- [2] Kong, L., Niu, Q., & Yang, P. (2023). *Design and Implementation of UART Based on Verilog HDL*. Highlights in Science, Engineering and Technology, TPCEE 2022, Vol. 38, pp. 949–955.
- [3] IEEE (2017). *IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language* (IEEE Std 1800-2017). IEEE.
- [4] Kashyap, B. & Ravi, V. (2020). *Universal Verification Methodology Based Verification of UART Protocol*. Journal of Physics: Conference Series, 1716(1), 012040.
- [5] Fang, Y. Y. & Chen, X. J. (2011). *Design and Simulation of UART Serial Communication Module Based on VHDL*. 2011 3rd International Workshop on Intelligent Systems and Applications.
- [6] Ciletti, M. D. (2011). *Advanced Digital Design with the Verilog HDL* (2nd ed.). Pearson Prentice Hall.
- [7] Mentor Graphics (2020). *ModelSim SE User's Manual*. Mentor, A Siemens Business.
- [8] Kamath, A., Mendez, T., Ramya, S., & Nayak, S. G. (2022). *Design and Implementation of Power-Efficient FSM based UART*. Journal of Physics: Conference Series, 2161(1), 012052.

The Demo Video of Transmitter :

<https://drive.google.com/file/d/1uSLhzyrSkQOC76BSGrjFN7GPY1uvPuXz/view?usp=sharing>

The Demo Video of Receiver :

<https://drive.google.com/file/d/1TyqJy39cPINKaFIDVvVuSGR1lt1xYdkr/view?usp=sharing>